

or data structure knowledge. This chapter guides you through developing this crucial skill.

How to Understand Python Error Messages

When they're confronted with an error message's large wall of technobabble text, many programmers' first impulse is to completely ignore it. But inside this error message is the answer to what's wrong with your program. Finding this answer is a two-step process: examining the traceback and doing an internet search of the error message.

Examining Tracebacks

Python programs crash when the code raises an exception that an `except` statement doesn't handle. When this happens, Python displays the exception's message and a *traceback*. Also called a *stack trace*, the traceback shows the place in your program where the exception happened and the trail of function calls that led up to it.

To practice reading tracebacks, enter the following buggy program and save it as *abcTraceback.py*. The line numbers are for reference only and aren't part of the program.

```
1. def a():
2.     print('Start of a()')
❶ 3.     b() # Call b().
4.
5. def b():
6.     print('Start of b()')
❷ 7.     c() # Call c().
8.
9. def c():
10.    print('Start of c()')
❸ 11.    42 / 0 # This will cause a zero divide error.
12.
13. a() # Call a().
```

In this program, the `a()` function calls `b()` ❶, which calls `c()` ❷. Inside `c()`, the `42 / 0` expression ❸ causes a zero divide error. When you run this